

CHAPTER

2

Authentication and Session Management

Authentication is the front gate of any secure web application. Authentication is used to establish the identity of your users, and without it, other functions of the site, such as authorization, cannot operate. The topic of authentication includes the login process, session management, password storage, identity federation, and several other subtopics. This chapter explores what you as a developer need to focus on when building a secure authentication process. We will provide several secure coding techniques around building a login mechanism, leveraging open source authentication mechanisms, session management lifecycle, storing and managing passwords, as well as other techniques needed to provide a secure authentication mechanism for your website. Let's get started!

Registration of New Users

For consumer-centric websites, your users' first interaction with your authentication process will be when they register an account with your website. Your registration workflow should, at the very minimum, request the username, email address, and password for the user. A display name may also be required. The user's email address should be verified before the account is fully enabled. Financial websites may require additional information like savings account number or credit card verification workflows. If you support multi-factor authentication, a user's phone number (for SMS multi-factor) or some other setup process for a multi-factor mobile or native application may be required. If the "forgot password" or other authentication workflow requires the proper answering of one or more security questions, now is the time to prompt the user to set them up as well.

Who Should Build Authentication for Your Project?

Authentication is an incredibly important security layer in your application. It is critical that you do not "reinvent the wheel" every time you build a new website. Especially for large enterprises, we hope that your most senior architects build the authentication layer and then reuse it on other projects. Even better, we hope that large organizations provide a well-secured and well-maintained authentication *service* that all company websites (not just Java-based) can utilize. If you find yourself building an authentication layer in your application, there better be a good reason to do so! Make sure you are not reinventing the wheel; make sure you are not rewriting a layer that already exists within your organization.

Preventing Automated Registration

Especially for popular and high traffic websites, attackers often use automation to register a large number of accounts for malicious purposes. This is often the only area where we recommend the use of a CAPTCHA,¹ but only after repeated automated attempts are detected. Ideally, we never want to force real users to resolve a CAPTCHA because their usability is poor, at best, when they are built in a secure fashion. CAPTCHAs are overall a fairly weak security control. Luckily there are several alternatives to CAPTCHA that accomplish the same defensive task without harming

usability.² A small time delay (i.e., connection throttling) can often achieve the same effect of minimizing malicious automated registrations. After a user is created, your application should enforce a time limit of 15 minutes before another user can be registered from the MAC address or IP address. These and other techniques can significantly slow registration bots.

One technique that may help stave off automated registration abuse is to add an additional form component, such as a text field, and then hide it with CSS by positioning the component off-page or use some other method to make it invisible to users. A normal user would never see or be able to fill this “hidden” text field with any data.

Image

Registration bots will often automatically prefill these form fields because they are often not aware of CSS, or at least cannot easily recognize when style hides a form component. When processing registration submissions on the server, if this hidden parameter is anything other than an empty string, you know it is from a bot and you can safely ignore and log that registration submission. (Proper logging of security-related events is discussed in [Chapter 9](#).)

The Basic Flow of the Login Process and Session Management

Before we describe attacks against authentication in detail, let’s begin by discussing the basic flow of how a login process should work. As shown in [Figure 2-1](#), there are many steps to consider, such as when to use HTTPS, how to manage the session and timeouts, how to handle credentials, and how to end a session. Let’s get started!

Image

FIGURE 2-1. *Login and session management workflow*

Login Workflow Step 1: Anonymous Session Created on First Hit

When a user first visits your website, JEE automatically creates a session for them and sets a JSESSIONID cookie even before that user logs in. Tracking a user’s anonymous behavior is appropriate and useful, especially for eCommerce websites. For example, you may wish to track which products a user has looked at so you can provide targeted advertising or otherwise change and improve the user experience.

It’s important to note, however, that when you support anonymous sessions before a user logs in, an attacker can easily generate an active session ID at any time. If an attacker can trick a user into using a known session ID, it can lead to a very dangerous situation called *session fixation*, which we discuss in detail shortly.

Login Workflow Step 2: Starting HTTPS and Encryption in Transit

As we discuss in [Chapter 6](#), HTTPS provides the benefits of *confidentiality* (a spy cannot see your data), *integrity* (a spy cannot modify your data), and *authenticity* (by the power vested in the certificate authority system, browsers can verify if a user is visiting the right site for that domain). With that in mind, is it safe to deliver the login form (the “login landing page”) over HTTP, as long as you securely submit the password over HTTPS? The answer is *no*. It is critical that you deliver login forms over HTTPS. Why? Many sites deliver login forms over HTTP but protect the password over HTTPS, but this is a very bad practice.³ Consider this basic login form:

Image

What is the harm if an adversary can see this data in transit? Absolutely none! All of this information is publicly available and an attacker could simply load up the form in his own browser and inspect it to his heart's content. HTTPS is more than just *confidentiality*; it also provides *integrity*. However, consider the harm attackers could do if they can modify your login form. That harm is significant! An attacker could insert a keylogger, change the form HTTP POST action to submit your password to a hacker-controlled website, or inject JavaScript to exfiltrate the password to another hacker-controlled website when you click Submit. Any sensitive form should be delivered over HTTPS, especially the login form.

You also need to maintain HTTPS for all content any time a user is logged in to your web application. HTTPS will protect the session identifier and other sensitive data in transit. HTTPS also enforces integrity. It will prevent the attacker from modifying your web pages because content cannot be injected into your website if it is being delivered over HTTPS. The signed SSL certificate also positively identifies the source of a web page's content. This makes man-in-the-middle attacks very difficult without access to a stolen SSL certificate.

Image

TIP

Make sure to force your entire website to HTTPS from the moment you deliver a login page until the user is done logging out!

A properly configured and updated SSL server is one of the best defenses that every website should have. One of the best resources to check your SSL configuration is Qualys SSL Labs, run by Ivan Ristic.⁴ The site offers a free suite of tests to detect any misconfigurations in your server's SSL setup.

Login Workflow Step 3: Processing and Verifying Credentials

At this point, we assume that your user is already registered as a user in your web application and has entry in your database or identity service, such as an LDAP server. At the very least, the subject's username and password ciphertext should be stored in your identity persistence mechanism. Password storage is a complex topic, which we discuss shortly.

In the previous step, the user submitted his username and credentials over a login form via HTTPS. The username is first looked up in the database to ensure it exists. If it doesn't, you give the user a generic failure message such as "username/password combination is invalid." You next hash the submitted password using the same algorithm and salt you initially chose to store the password.⁵ If the user's password ciphertext matches the ciphertext that is stored in your database (and hopefully the multi-factor value they entered is also valid), then the user has logged in successfully and you can start their session.

Login Workflow Step 4: Start the User's Authenticated Session

Because HTTP is a stateless protocol, you have no way of knowing who the user is between requests without extra effort. Fortunately, you have a tasty way of adding state to HTTP—cookies! As soon as a user logs in to your website, you first want to invalidate the current anonymous session and create a new authenticated session. This is done to stop session fixation, which occurs when an attacker uses a known anonymous session ID to access a user's personal data after they login. The session is maintained by HTTP cookies that are delivered from the server and are saved in your browser. To make your job easier, most Java web

frameworks will automatically create session cookies for you simply because your code has created a new session:

Image

You should ensure that your website does *not* support URL rewriting, or modifying URLs to add the session identifier to the URL directly, something that most Java frameworks support by default for legacy purposes. If you have ever seen a URL like this, you know that URL rewriting is in use:

Â Â Â Â Â Â Â <https://www.somesite.com/myAccount.jsp;jsessionid=902B45163DCCFAE90748DDB358CC7502>

URL rewriting is a feature of some servers that occurs when subjectsâ€™ browsers do not support cookies or cookies have been disabled. URL rewriting is a security vulnerability because it places your session ID at risk of exposure in browser history, web server and proxy logs, and referrer headers. You want to avoid placing any sensitive data in the URL, especially session identifiers. Rather than degrade to this less secure option, your website should instead inform that user that cookies are necessary for website operation. To ensure that your web application does not support session rewriting, you can force the session to *only* support cookies via the following simple addition to your web.xml configuration file:

Image

This tells the server that your application should only use cookies to track the session and not URL rewriting. Additionally, if you use Apache Tomcat, you can set the following in your server.xml or context.xml configuration file to disable URL rewriting at the server level:

Image

Login Workflow Step 5: Do Cool Things

Now that your user is successfully logged in, she will start doing â€œcool thingsâ€ with your website, such as blocking her mother when she shares embarrassing baby photos over social media, buying Fra-Gee-Lay fishnet stocking novelty lamps for the office, or commenting on cute cat videos. You know, the important things. With each request, the browser will automatically include the authenticated session cookie, and the server will validate that the cookieâ€™s session ID refers to an active session on the server. This is the heart of how authenticated requests are validated. Consider the session ID to be a temporary credential, like an ID badge that is only good for one day, which admits the user into more secure areas of a site. As such, great care should be taken to protect the session ID because if it is lost or stolen an attacker can use it to gain access to any functionality that the legitimate user can.

Login Workflow Step 6: Potential Re-Authentication for Sensitive Operations

Some requests from your users will be highly sensitive and should require an additional authentication security check. Some classic examples of sensitive transactions that should require re-authentication are changing your password or email address, system administration, privilege assignments, and high-value money transfers. A secure change-password feature will not only require the new password and confirmation, but should also require the current password. This is done to prevent an attacker who has hijacked a userâ€™s session (often by just sitting down at the victimâ€™s terminal after they leave) from changing the userâ€™s password and thus fully compromising the account. This also helps to mitigate CSRF class attacks in addition to minimizing the effect of various forms of session theft.

Login Workflow Step 7: Idle Timeout

Because a user's session ID is a highly sensitive piece of data that can be used to compromise a victim's account if stolen, you want to ensure that the *life* of the session ID is limited as much as possible. If the user is inactive for a certain amount of time, you should invalidate that session on the server and force the user to log in again starting with the user's next request.

Java EE has a default session timeout mechanism supported by most Java web frameworks. In `web.xml`, the `session-timeout` directive will force sessions of inactive users to invalidate server-side once users are inactive for 20 minutes.

Image

Many modern AJAX-centric web applications, however, contain pages with auto-refreshing mechanisms. These pages may auto-update every few seconds, meaning that the server will never see the user as *idle*, and thus sessions could continue indefinitely. If possible, consider stopping the auto-refresh mechanism via JavaScript when the application is not the currently active browser tab. You should also be able to detect via JavaScript on the client when the user is inactive for a certain amount of time, even when the auto-refresh is active. Once this client-side timeout is reached, the JavaScript should execute a GET request to a page on the server that will invalidate the session and return the user to the login screen. Note that you still want to support server-side idle timeout, but this type of client-side idle timeout can significantly improve a site's security posture, especially for users who leave their browsers open for long periods of time.

Login Workflow Step 8: Absolute Timeout

When determining the life of a session identifier, you not only want to limit the amount of time the user can be idle, but you also want to limit the total length of an authenticated session by closing it and forcing re-authentication after a certain period of time. This prevents an attacker who has compromised a session from keeping it active indefinitely by using an automated process to simply hit the site every few minutes. Setting a limit for how long a session may be active is called *absolute timeout*. Absolute timeout is poorly supported in most web frameworks overall, and absolute timeout support is only now emerging in the Java ecosystem.

Implementing absolute timeout in your Java web application is straightforward; simply create a Java EE filter that provides the following logic:

Image

This logic simply says that when a user makes a request, if the current time minus the login time (the current length of time the user was logged in) is greater than the maximum amount of time a session can be active for, then invalidate the current authenticated session. Depending on your application logic, this could then force the user to log in again, or simply take them to an *anonymous* portion of the site. This filter is only triggered on request and does not time out the session automatically. That is acceptable in combination with idle timeout because idle timeout will trigger automatically when the user is inactive.

Login Workflow Step 9: Logout

Another critical way to limit the life of an active session is to encourage users to log out when done so you can immediately terminate their session on the server. A logout button should appear prominently on your website, conventionally in the upper-right corner of every page in your site.

When clicked, the logout button should send a request to the server that triggers logic, which will immediately destroy the current active session by a call to `HttpSession.invalidate()`.

Attacks Against Authentication

This section will explore various ways that attackers will harm your login or session management mechanisms by achieving access without the proper credentials or permission. We will also explore various ways to defend against these common authentication layer attacks. Let's dig in!

Session Hijacking

One of the most significant risks to your authentication layer is the risk of session hijacking. When an attacker can guess or steal your session ID, when the attacker can force a victim to use a specific session ID, or even something as trivial as when a victim leaves him or herself logged in to an account on a public computer, the threat of session hijacking surfaces.

The ways in which hackers compromise your session are many. As discussed in [Chapter 4](#), session hijacking via cookie theft can occur through cross-site scripting with attacks as simple as the following:

Image

Session hijacking can also occur if you fail to use HTTPS anytime throughout the authenticated session lifecycle. Eric Butler released a session hijacking open source Firefox plug-in called "Firesheep," which swept the security world in late 2010.⁶ Firesheep allowed anyone to hijack authenticated sessions simply by being on the same network with the victims. Most major email and social media sites at the time defaulted to insecure transport (HTTP) after authentication. Even today, at the time of this writing in March 2014, Google has just forced its Gmail service to only support HTTPS for the first time.⁷ Since 2010 HTTPS has been the default option.

Even poorly configured HTTPS (due to weaknesses exposed because of weak ciphers, short key lengths, or compression) allowed savvy attackers to decrypt session identifiers via methods such as the CRIME attack⁸ even when the transport is encrypted! Other problems such as weak or predictable session IDs can also lead to session hijacking.

Image

TIP

If you are a Tomcat user, consider increasing the size of your session ID! A longer session ID is harder for an attacker to guess. Increasing the length is as simple as editing `$CATALINA_HOME/conf/catalina.xml` and adding `<Manager sessionIdLength="32" />`. The default value is 16 characters; consider doubling it to 32!

Session Fixation

As discussed, a specialized form of session hijacking is called session fixation. Session fixation occurs when an attacker can trick a victim into logging in to a site using a session ID that is known to the attacker. The attack scenario works like this:

1. First the attacker visits your website to obtain a valid session ID.

2. The attacker then creates a URL for the vulnerable website that includes this session identifier as an HTTP GET parameter value, for example using a URL rewriting vulnerability.

3. The attacker tricks a victim into clicking on the URL to visit the vulnerable website.

4. The vulnerable website recognizes the session ID in the URL and resumes the session that was started by the attacker.

5. Once the victim logs in, the attacker is now able to compromise the victim's session because the attacker knew the session ID before the victim even pressed the link!

As we described, it is important to support session management through cookies only and to disable URL rewriting. However, URL rewriting is not the only way to preset a user's session ID. Cross-site scripting can also be used to set the session ID. An attacker can even sit down at a user's workstation and manually edit the user's cookies with a tool such as Firebug. The only real way to prevent session fixation is to regenerate the session once the user successfully logs in:

Image

Image

TIP

After your user successfully logs in, invalidate the current session and create a new session for the user!

Secure Cookie Properties for Session Management

Since HTTP cookies are the primary way that most web applications save and transport the session identifier between the browser and your server, it is important to protect them as much as possible. There are two session cookie properties that are critical to good web application security. These are the *Secure* cookie flag and the *HTTPOnly* cookie flag. Secure cookies will only be transmitted by the browser via an HTTPS connection. Combine Secure cookies with HTTPS and Strict Transport Security and it becomes impossible to leak session cookies over a plain-text transport. The HTTPOnly flag prevents JavaScript from accessing cookie data, but still allows the browser to automatically attach cookies to requests that are made to the appropriate domain and path. HTTPOnly cookies are one very important way to minimize session theft over cross-site scripting, but you still need a comprehensive cross-site scripting defense strategy, as we discuss in detail in [Chapter 4](#).

Luckily, most Java Servlet and JEE containers set the Secure and HTTPOnly cookie settings by default after session creation. Be sure to verify that this is the case with your application server by monitoring the traffic with an intercepting proxy such as Burp Suite. You should see the Secure and HTTPOnly flags (in **bold**) in the Set-Cookie response headers when the application sets a cookie:

Image

If your application server is not setting session cookies with these flags automatically, you can specify that they be used in your web.xml deployment descriptor:

Image

You can also set cookies to use these flags programmatically. This is especially useful in cases where your application needs to set other cookies beyond the JSESSIONID:

Image

Dangers of Storing Sensitive Data in Cookies

We also want to encourage you to avoid the use of cookies to go stateless by storing session data such as the user ID, roles, and other information that identifies a user. Even if the data is encrypted, attackers who steal your cookies can conduct a replay attack by simply placing the cookies in their browser and visiting the victim site. Even if the attackers cannot decrypt the cookies, they can still reuse them, visit the vulnerable website, and compromise the session. Also, as we discuss in [Chapter 6](#), getting cryptographic storage right is incredibly difficult. Even the world's best experts in applied cryptography get it wrong on a regular basis.

Credential Security

The user's password and other credentials are the keys to the kingdom for that user. Your site should *never* return a valid password in any response. When a user's password is stolen from your website, it might not be only your website that is affected. Unfortunately, many people reuse their passwords on multiple sites, so if a password is divulged on your site that user's entire online life may be compromised. Also, weak passwords can be compromised easily through a variety of different attacks. You need to ensure that your password policy is strong enough for security without harming usability, you need to store your users' passwords securely, and you often need to provide secure secondary password services such as the forgot password workflow.

Password Policy

Traditional wisdom on password policy is often out of date. We often see policies that enforce at least one uppercase letter, one lowercase letter, a number, and a non-alphanumeric character. How many corporate security policies would allow "Password1!" to be an acceptable password? It is ten characters and it contains all the necessary special characters, but is it secure? No way! In fact, "Password1!" is one of the most commonly used passwords in business, with "Password1!" not far behind in the "worst password" category. Although it's still important to enforce special types of characters in passwords, it is just as important—if not more so—to ensure that your users are not using passwords that contain dictionary words and to ensure that they are not using commonly used passwords, even ones that would pass traditional password policy. Even better, do not depend on passwords at all because the era of passwords as a single authentication factor is dead. It's time to shift all of our systems to multi-factor authentication.

Password Managers

Another aspect of password policy that is critical in the modern era is to allow large complex passwords. This helps enable tools such as password managers. A password manager is a piece of software that will automatically create a long, random, and complicated password for each website a user visits and save those passwords securely. Bob Lord, the Corporate Security Officer at Twitter, forces all employees to use password managers.⁹ Although this does not make the news like exciting hacks do, it is a huge win for Twitter in terms of securing the organization. Be sure your website is ready for password managers!

In order for your website to be “password manager ready,” we suggest that you allow your users to choose very long passwords, but not too long.¹⁰ Also, be sure to avoid complex JavaScript on your authentication form fields because that could throw off a password manager’s ability to work with your website. Lastly, it is important for your website to allow users to paste into the password form field (this can be disabled via JavaScript).

Password Storage: Verify but Not Recover

How should you store your passwords? Let’s start with a few basic requirements. Essentially, you want to be able to verify, incredibly quickly, if a user’s password is correct. However, even if attackers should somehow steal your database or credential data, you do not want them to be able to read the passwords, within any reasonable amount of time, even when supercomputing capabilities are available. The best password storage strategies use password-based key derivation functions that store the password into a form that is unrecognizable from the original. This prevents even website administrators from knowing their users’ passwords! There are also working groups of cryptographers researching and building next-generation password hashing algorithms.¹¹

The best (and possibly only) threat model created on password storage is from John Steven.¹² We challenge the application security community that disagrees with John’s assertions to publish a competing threat model, or to critique his work in some scientific and publishable way. Plenty have disregarded these recommendations without providing any sort of scientific evidence as to why.

That aside, the whole purpose of strong password storage is to prevent attackers from discovering your users’ passwords even when your database and/or password ciphertext is stolen. Once your password data is stolen, attackers will attempt to discover your users’ passwords “offline”: using dictionary attacks, brute force attacks, and rainbow table attacks (i.e., precomputed databases of hashes).

A brute force attack against a password store occurs when an attacker walks through a list of every possible password, creates the ciphertext for each potential password, and compares that ciphertext to the passwords stored in your database. Although this seems like an inefficient attack,¹³ attackers now have access to both specialized password cracking algorithms and high-performance GPU password cracking rigs that can attempt many billions of password attempts per second against your password storage mechanism.¹⁴ Attackers also have large lists of known passwords, which are often the first ones they try because people tend to reuse the same passwords on multiple sites.

Image

TIP

A great way for an attacker to try a brute force attack against your password store is to simply try many password combinations against your own site’s login process! This has the side benefit (to the attacker) of using your own hardware! Make sure to set a maximum number of incorrect tries before you lock the account. Multi-factor authentication is also a good defense that keeps brute force attacks at bay.

A dictionary attack against password storage occurs when:

Image An attacker has stolen your stored password ciphertext.

Image The attacker understands your password storage algorithm.

Image The attacker attempts to discover passwords by using a large list of commonly used password and dictionary words, generating the ciphertext for that password, and then comparing it with your stored ciphertext. Once the attacker finds a match, they have discovered a password!

Attackers can also use large precomputed lists of hashes called Rainbow Tables. In 2012, LinkedIn was compromised and had password data stolen.¹⁵ At the time, LinkedIn was using the MD5 hashing algorithm for password storage. This was a dangerously bad idea because flaws had been discovered in MD5 as early as 1996 and even its own author stopped recommending using it as of 2005.¹⁶ Hackers were able to discover over 90 percent of LinkedIn users'™ passwords within the first day after the compromise by simply looking up the password hashes in a database. In fact, there are dozens of websites that host free opensource MD5 databases, and you can even find the original plain-text of many MD5 hashes by googling.¹⁷

So what kind of password storage is resistant to an attack of this nature? Well, first of all, if you do not enforce a strong password policy, any password storage mechanism is useless. We discussed good password storage mechanisms previously.

The first go-to algorithm for password storage in the Java ecosystem is the natively supported PBKDF2 NIST Standard KDF (Key Derivation Function).¹⁸ This function is both one-way and can be configured to process relatively slowly"on purpose. Why would you want to use a slow algorithm for password storage? GPU cracking rigs and other supercomputing resources can attempt many billions of hashes per second. When using a purposely slow algorithm with the proper configuration settings (i.e., work factor or iteration count), those billions of attempts per second become mere thousands of attempts per second on the same hardware. What used to take days or weeks to crack now will take months or years to crack on the same hardware. No password storage mechanism is perfect. The best you can hope for is to buy time so you can recover from the data loss incident. You may need to send some embarrassing emails to your user base warning them to change their passwords, but with luck, nobody will have their actual accounts compromised. Slow (adaptive) algorithms help us in this endeavor.

The following is the proper way to hash a password using the PBKDF2 algorithm in Java.

Image

The preceding password validation function, when set at 120,000 iterations and a 512 bit key length, takes approximately 750 milliseconds to execute on a laptop with a 2.3 GHz core i7 processor and 16GB 1333 MHz DDR3 RAM. This achieves our goal of being slow enough to hamper brute force attacks, while still being fast enough to not harm the user experience for a legitimate user with the correct password. As time goes by, computers will become faster and this iteration count will take less time to complete. This function can be modified to be stronger by increasing the number of iterations. Since the iteration count is stored in your database along with the hashed password, you can increase the number of iterations over time without harming backward compatibility.

Image

TIP

Use PBKDF2 to store the password of each user. Use a different random salt for each user. Use an iteration count that is as high as you can tolerate! The larger the iteration count the more secure your password storage system is.

Another option for password storage is the *scrypt password*-based key derivation function. Not only is scrypt slow but it can also be memory hardened. That is, scrypt can be configured to consume a certain amount of memory when executed. This will limit the attacker's ability to discover the stored password using more sophisticated hardware. Although Java does not provide native scrypt support, the open source Java cryptographic provider, Bouncy Castle, does indeed support scrypt,¹⁹ albeit in an unpublished way as of March 2014.²⁰ Instructions for installing Bouncy Castle can be found at <http://bouncycastle.org/specifications.html#install>.

Our recommendations of PBKDF2 and scrypt are formal Key Derivation Functions (KDFs). They are made to take a password and (slowly) churn out a key. These algorithms are the best ways to store a password because they are well-tested, standard algorithms. You should not try to write your own implementation unless you have a PhD in applied cryptography.

Even though PBKDF2 and scrypt are good choices for password storage for most types of websites, when was the last time you were told to actually *slow down* your code? These algorithms can require an extensive amount of hardware in order to support a large number of concurrent logins. In fact, the Django open source Python content management system suffered from a well-publicized denial-of-service vulnerability because it used the PBKDF2 algorithm and supported unlimited length passwords at the same time!²¹ By sending very long passwords, attackers could force the systems running Django to consume an inordinate amount of processor resources. A 1 megabyte password submission could take up to a minute or more of CPU time! Do not make the same mistake. Although you should support long passwords, and PBKDF2 is a reasonable algorithm to pick for password storage, do not support *extremely* long passwords. Ultimately, the developers of Django settled on a 4096 character maximum password length. This seems like a reasonable cap²² for now.

^ ^ ^ ^ ^ ^ ^ Image

^ ^ ^ ^ ^ ^ ^ **TIP**

^ ^ ^ ^ ^ ^ ^ PBKDF2 is a good choice for password storage in the Java ecosystem. Be sure to add a unique per-user salt of at least 32 characters in length. Be sure to make the iteration count as large as possible without harming your user experience. A good rule of thumb is to set the iteration count high enough so that processing the PBKDF2 hash takes to up to one-half second on your production server and to increase the iteration count over time as faster hardware becomes available.

Password Storage and Handling Scale

So what do you do when you need to handle the password verification and concurrent login process for a large number of users? You can certainly throw a lot of hardware at the problem and call it a day. But what if you want to have secure password storage, and be able to handle a large number of concurrent logins with efficient use of hardware? That is where a *Keyed-Hash Message Authentication Code* (HMAC) comes in. When properly implemented, HMACs can accomplish secure password storage at a massive scale and still make efficient use of hardware. Most companies that need a massive scale (big banks, and so on) already have a good key management infrastructure in place. The strength of an HMAC is governed by the strength of the underlying hash functions, the size of the key, and the ability to keep the key secret. HMACs are essentially as fast as a hashing algorithm and will not be reversible if the key is managed properly. However, if the key is ever stolen or lost, the whole system is broken (because the password cracking problem is reduced to a simple hash function). When building a password storage system, KDFs such as PBKDF2 and scrypt are a good first choice. However, we have seen large companies that use scrypt for their password storage system consume a significant percentage of their server farm resources just for password processing during peak load! Yeah,

crazy. So by moving them to an HMAC solution, and possibly even making custom hardware for their servers,²² you are able to radically save them money on hardware utilization while still providing a secure password storage system. Yeah, baby!

A system that uses HMAC is a lot more difficult to engineer for secure password storage. You need to isolate the processing of the password to truly protect the key. The ideal system is a web service that takes a plain-text string and spits out the HMAC hash without ever exposing the key to your application code. This HMAC web service should be isolated on separate hardware so that even if your database, file system, and web server are all compromised, your password storage mechanism is *not* compromised. Even if your entire database is compromised, at least the other websites where your user reuses his or her password are not also compromised.

Salt

Regardless of which algorithm you choose for password storage, be sure to add a unique per-use *salt* to your user's password before protecting it. Because PBKFF2 and scrypt produce predictable output, if two users have the same password, their password ciphertext will be the same. For example, the SHA-256²³ hash of "password1" will always be:

Image

If your password store is ever compromised and an attacker is able to reverse one of the hashes, the hacker could simply look for other entries that have the same value and be able to compromise many accounts for the "price" of one.

Adding a random salt value to your hash ensures that no two password hashes are the same, even if they use the same starting value. The following SHA-256 hashes all used the same starting value of "password1," but this time with a random salt applied:

Image

This also radically increases the size of the rainbow table that an attacker would need in order to store all precomputed hashes of all possible passwords. Adding a 32-character random salt would mean that instead of one entry, an attacker would need 62³² or 2.272+e57²⁴ entries, just for "password1"!

Whenever a user creates or changes his or her password, create a 32-character random value, the salt. Then, add the salt to the password before you protect it with PBKDF2, scrypt, or your specialized HMAC setup. Be sure to store the salt along with your hashed password in the database so that you can use the salt again on subsequent authentication attempts. Without the stored value of the salt, you will never be able to recompute the correct hash to check if a password the user entered is correct!

We especially want to thank John Steven for his incredible work and research on password storage. For more information, please check out his OWASP Password Storage Cheat Sheet.²⁵

Forgot Password Workflow

If you are encouraging your users to not reuse passwords, and you enforce a strong password policy, sooner or later you are going to have to deal with users forgetting those long complex passwords. The most common way that consumer-centric websites support the "forgot password" workflow is through a password reset form that asks for a username and then sends an email with a one-time link to the email address on file for the account. If you are a user of

online banking, try to reset your password. You will note that most financial institutions do not trust the security of email as a fundamental part of the forgot password workflow. They have a good reason for this: Email was not designed with privacy or security in mind.

If possible, do not support the forgot password workflow as part of your website. Force your enterprise users to actually call in or use some other well-vetted manual process. This alone will deter many attackers. While this is ideal, it does not scale very well. If you must have an automated forgot password process, consider the following example from the banking industry:

1. Establish the user's identity with requests for two or more pieces of information such as the user's Social Security number²⁶ and account number. Start a session for this process and ensure that the rest of the process comes from the same session.

2. Once you have successfully established the identity of that user, as an optional step, consider asking the user a security question²⁷ that users set up during registration. Some banks use this; some do not.

3. Now that you have fully established identity, send a multi-factor token out of band. The best bet is to use a dedicated multi-factor device or application. Next best is to use SMS to the phone number on the account. The worst option is to send a token over email, but by having already established identity this is still better than an email password reset link alone.

4. Once you have verified that the user's token is valid, allow the user to reset his or her password to a new, strong value according to your organization's password policy.

For more information on this topic, please see the OWASP Forgot Password Cheat Sheet by David Ferguson.²⁸

Image

TIP

Make sure that even when a user's email address is compromised that forgot password workflows cannot be successfully attacked!

Username Harvesting

Another type of authentication-layer attack against your web application is *username harvesting*. This attack occurs when threat agents attempt to verify if a username is valid or not. One simple way to conduct this attack is to inspect the error message when login failures occur. Suppose you attempt to log in with a bad username and a bad password and the error message says, "The username entered is not valid." This clearly tells the attacker that the attempted username is indeed not valid! Now suppose you attempt to log in with a valid username and a bad password. What will be implied if the error message says "The password you entered is not valid"? This tells the attacker that the username entered is indeed valid. Any time your error messages are different when the username is valid or when the username is not valid, attackers have an easy means to conduct username harvesting. Many secure coding standards encourage your error message for login failure to be the same regardless of the type of failure to protect against username harvesting. For example, regardless of whether the username entered was valid, your application could return the generic message "Invalid username/password combination."

Simply changing the text of the message is not enough. For this to work properly, you need to be very conscious of your actual login logic and even formatting. Consider the code in [Figure 2-2](#), with nonprintable characters shown.

Image

FIGURE 2-2. *Code sample with nonprintable characters*

Even though the exact same error message is used in both conditions, an intercepting proxy such as Burp Suite can easily detect differences in output that would appear identical on visual inspection (see [Figure 2-3](#)).

Image

FIGURE 2-3. *Burp Suite illustrating differences between error messages that normally appear identical on visual inspection*

However, it's pointless to protect against username harvesting if you allow users to select their own username during account registration. Consumer-centric websites almost always verify if a requested username is in use already and disallow that choice if a duplicate name is selected. This can easily be leveraged by an attacker to harvest valid usernames. In fact, some user registration mechanisms provide an AJAX-like widget to verify if a requested username is valid or not. An attacker working with a list of names and email addresses can leverage these features to quickly harvest a large number of valid usernames. Consider implementing a throttle to slow down automated requests to these mechanisms. Some enterprises and banks, such as HSBC, even force complex policy requirements on usernames, or create complex machine-generated usernames, something worth considering for secure web applications.

Brute Force Attacks, Account Lockout, and Multi-Factor Revisited

Once an attacker knows a valid username, he can attempt a brute force attack. A brute force attack occurs when an attacker attempts many passwords in high volume against a victim's account, attempting to discover their authentication credentials through trial and error. Old school brute force tools such as *Brutus* make brute force attacks simple. As the name implies, this is not a savvy or intelligent attack.

There are many ways to counter brute force attacks. The most common defense is *account lockout*. Account lockout is a defense by which, after an attacker attempts to incorrectly log in to an account a certain number of times, the account will be disabled temporarily preventing anyone from logging in to the account. Account lockout schemes are often triggered after just a few failures. As you force your users to create more and more complex passwords, forgetting a password or entering it incorrectly will only be more common. Setting your account lockout to ten failures will trigger the defense against automated attacks quickly but will still provide users with significant leeway to enter their password correctly. If your account lockout mechanism triggered too quickly, your website would certainly be secure but with greatly hampered usability. You need to find a balance. The key words here are *preventing anyone*. If an attacker has knowledge of various usernames in your system, account lockout can be used to conduct a *denial of service* attack against your web application, effectively stopping any user from logging in to their account.

Another form of account lockout is a reverse bruteforce attack. A reverse brute force attack occurs when an attacker attempts a commonly used password once against many different accounts in your system. The benefit of this attack is that it will not trigger account lockout against any one account. Disallowing commonly used passwords, even ones that fit your normal password policy, is a good mitigation to this attack.

In general, we recommend keeping away from account lockout if your user has enabled, or if you force, multi-factor authentication, which we discuss shortly.

Remember Me Feature

Software developers are often tasked with supporting the “remember me” login feature. This feature is essentially a wrecking ball to all that is good and just within an authentication layer. In order to support the “remember me” feature, you essentially need to keep the user logged in for days, weeks, or months, something that is in direct opposition to everything we have discussed about properly limiting an authenticated session. Also, “remember me” features are often subject to a *replay attack*. They often require that you store a credential or a session identifier in a cookie with no expiration. Even if credentials stored in a cookie are protected with the best crypto on the planet, an attacker who steals your cookies can replay that encrypted cookie in his own browser, use *your* server for decryption, and compromise the account. If the business forces you to support a “remember me” feature, consider just remembering the username and prefill it on the login screen. Avoid implementing “remember me” that stores the users’ credentials or forces you to maintain an authentication token or authenticated session that is active for a long period of time.

Multi-Factor Authentication

Please take a few deep breaths, center yourself, put down the book, and repeat after me: *The era of the password as a sole authentication credential is dead*. The most fundamental feature you can support for a secure authentication mechanism is multi-factor authentication. Multi-factor, or multi-step, authentication involves requiring two or more factors at login time (as well as during re-authentication). That includes something your user *knows* (such as a password) in addition to something the user *has* (such as a multi-factor token or mobile application) or something the user *is* (such as a retinal scan or a fingerprint).

It is fairly straightforward to implement multi-factor authentication. Almost every major web application already supports it today or is moving in that direction. Multi-factor authentication reduces the impact of password theft, mitigates bruteforce attacks without the need for account lockout, and in general protects the front door of your web application in a way that is much stronger than username and password alone.

Seed Storage

First of all, who is storing a copy of your multi-factor token generator seeds? These seeds are used to keep the multi-factor token value in sync between the token and the server. If stolen, the seeds can be used to generate valid multi-factor token values. In March 2011, RSA was breached by a hacker who stole the multi-factor token seeds from their SecurID product. Three months later RSA admitted it was breached and that information used in the breach was used to hack Lockheed Martin and that it would be replacing SecurID tokens for nearly all of its customers.^{[29](#)} Consider solutions where third-party vendors do not control your authentication or multi-factor seeds!

Where Do You Send the Token?

After a user identifies himself during login, and the user's password is checked and proves valid (something the user *knows*), you next send a token or other secure message (the second factor) out of band. The user needs to verify that she is actually in possession of the configured device or channel (something the user *has*) in order to complete the login process.

Email is one potential avenue for token verification. Email verification typically takes the form of a link that the user must click, or a one-time code that the user must enter into the website in order to log in. Unfortunately, most email is not encrypted should not be considered a safe mechanism for data transfer. Email is the worst method for multi-factor authentication.

SMS is another option for token verification. Be sure to verify the account owner's phone number before depending on it for SMS verification as part of multi-factor authentication. SMS has the benefit of ensuring that your users are actually *in possession* of their mobile device in order to log in. There are also mobile applications, such as the one used by Twitter, that can be set to receive push notifications when the user attempts to log in. The user can then approve or deny the login request directly within the phone application without having to juggle numerical codes.

Of course, if someone's phone is lost or stolen, an attacker will have access to the authentication tokens. If a user's phone is stolen, will their first thought be to cancel access to their multi-factor accounts? Probably not, but at least this method requires attackers to perform extra steps and specifically target their victims instead of using a *shotgun approach* toward bypassing your authentication mechanisms.

Other mobile applications attempt to do away with the requirement to send and receive notifications from the server. These applications use an algorithm to generate a stream of tokens based on a timestamp and a pre-agreed seed that is set up at configuration time. They allow for multi-factor token generation where no data needs to be sent over the network from the server to the user.

The most secure multi-factor mechanism is a dedicated hardware token. The hardware token uses the same approach of generating a token on the fly based on a seed and a timestamp according to some algorithm. These tokens often require the user to enter a code in order to activate the token, which provides an additional piece of information that an attacker must know in order to log in. Unfortunately, users often balk at carrying extra devices and these types of dedicated hardware tokens are usually only used for the most secure of applications.

Federated Identity and SAML

SAML, the Security Assertion Markup Language, is a way to *federate* or *delegate* authentication and access control information between two different systems, often in different organizations. What does this mean and why would we use it?

Imagine that your website gains significant popularity. For example, you may have built a website that offers a new and clever way for people to buy specialized lamps and other lighting products. Now suppose that website reaches popularity of epic proportions. Now suppose a major company wants your website to give many thousands of their employees specialized access in order to purchase co-branded lamps for promotional purposes. This access may include very proprietary information (who are you selling lamps to?) that only company members should access.

Normally, your business partner would need to create and manage the accounts and passwords for these thousands of employees on your website. Every time a new employee is hired a new account would need to be created on your website. Every time an employee leaves the company

the account would need to be removed. For a company with thousands of employees this quickly becomes an administrative nightmare.

What if instead of your partner having to create and manage thousands of user accounts on *your* website, they could manage those accounts on their own systems? Suppose you had an arrangement with your partner (leveraging Single Sign On technology) that a user only needed to log in to your partner's authentication provider and, based on a prearranged "handshake," that user would automatically be authenticated to your awesome new lamp website because of your partner's assertion that the user is legitimate? This is the heart of what SAML and identity federation provide.

Unfortunately, SAML is neither simple nor truly standard. Each website or service you wish to integrate with will have subtle differences in how it needs to be implemented. Good partners will provide detailed requirements regarding how their SAML service works. This is an important technology that you will be confronted with in a variety of federation and Single Sign On enterprise services.

OAuth Basics

Open Authorization (OAuth) is a protocol that allows your web application to authenticate against other services on behalf of your users. This protocol allows your web application to authenticate to the OAuth service on behalf of your users without requiring you to store any credentials. If necessary, your web application can authenticate to another service without your users even being logged in to your application. For example, if you would like your website to "tweet" on behalf of your users, without ever needing to know (and therefore protect) your users' Twitter passwords or other sensitive information, OAuth is the protocol to accomplish this feat.

OAuth is a protocol that allows users of your web application to authenticate against other services without exposing any credentials to the application itself. For example, consider building a web application that "tweets" on behalf of its users. OAuth allows your web application to authenticate users directly on Twitter. Subsequently, your application is authenticated to the service and can perform any actions that the user authorized (such as tweet, read tweets, and so on). In this way, your application can now employ user accounts on another site, without requiring you to store any credentials or sensitive information at all.

Other Authentication Security Considerations

Here are a few additional steps to consider when building your authentication layer.

1. Attacks on SSL/TLS often target cookie values and therefore session identifiers. Consider frequent rotation of session identifiers when building custom session handling mechanisms or after re-authentication. The less time a particular session token is valid, the less time an attacker has to make use of it.

2. Provide your users with a prominent way to view their most recent logins with login time, logout time, IP address, and hopefully location. If users can see this information, they can help you spot a breach.

3. Notify your users via email when a password is changed, when multiple login failures occur, when their user profile or permissions change, or when other major changes occur to their authentication workflows.

4. In the user's profile, show all active sessions or federated relationships with other services and provide a way to shut down certain sessions or relationships, especially if you support OAuth, "remember me," or any long-lived authentication sessions.

5. Provide a capability to block logins from certain geographic regions.

6. Provide a capability to block logins during certain times.

7. Build a well-vetted centralized authentication service and/or library that many projects can leverage securely.

OAuth 1.0 was based on cryptographic roots but was vulnerable to session fixation and other issues.³⁰ OAuth 2.0 is currently in wide use and is used by many major consumer websites, such as Facebook, Twitter, and Google+. OAuth is also making headway in the enterprise for mobile authentication because of its ease-of-use.

Although OAuth is named Open Authorization (Access Control), it is still primarily an authentication standard. In most implementations, OAuth does allow users to grant permissions to apps to do things such as access data and friend lists, and post to social networks on your behalf. In this way, it is like the reverse of a traditional authorization scheme, as we discuss in the next chapter, because it enforces permissions for apps, instead of apps enforcing permissions for users.

For more information on the OAuth 2.0 standard, visit <http://tools.ietf.org/html/rfc6749>.

Additional Reading

One of our favorite resources on session management is the "Session Management Cheat Sheet" from the OWASP Foundation, written by Raul Siles.³¹ This is a good place to start before building a complex session management mechanism from scratch. Another cheat sheet in this series is the "Authentication Cheat Sheet," written by Eoin Keary.³² The Apache Shiro framework also provides a fairly elaborate and configurable authentication mechanism.³³

Summary

Authentication layer security is complex and requires precise design. Ensure that all authentication and subsequent authenticated requests are done over a secure transport. Secure the session correctly with proper cookie settings and timeouts. Force users to re-authenticate at critical application boundaries. Use PBKDF2, scrypt, or HMACs for password storage. Build secure forgot-password and change-password features. These are the essential security considerations for authentication and session management. You *can* secure the front gate!

¹ "Completely Automated Public Turing test to tell Computers and Humans Apart," <http://en.wikipedia.org/wiki/CAPTCHA>

² <http://textcaptcha.com/really>

³ www.troyhunt.com/2013/05/your-login-form-posts-to-https-but-you.html

⁴ <https://www.ssllabs.com/ssltest/>

⁵ In very secure environments, you should always run the password protection algorithm against the submitted password, even if the username is not valid. This will help prevent username harvesting by measuring the time difference in responses between entering a correct username, which protects the password, and an incorrect one, which doesn't. This extra time spent will also help slow down automated attacks.

⁶ <http://codebutler.com/firesheep/>

⁷ <http://gmailblog.blogspot.jp/2014/03/staying-at-forefront-of-email-security.html>

⁸ [http://en.wikipedia.org/wiki/CRIME_\(security_exploit\)](http://en.wikipedia.org/wiki/CRIME_(security_exploit))

⁹ <http://news.softpedia.com/news/Hack-in-the-Box-13-Twitter-s-Bob-Lord-Forces-New-Employees-to-Use-Password-Managers-344699.shtml>

¹⁰ <https://www.djangoproject.com/weblog/2013/sep/15/security/>

¹¹ <https://password-hashing.net/>

¹² <http://goo.gl/Spvzs>

¹³ Attempting to walk through every password of less than 10 characters, considering uppercase, lowercase, numbers, and basic non-alphanumeric characters would take approximately 90^{10} or $3.4867844e+19$ attempts!

¹⁴ <http://arstechnica.com/security/2012/12/25-gpu-cluster-cracks-every-standard-windows-password-in-6-hours/>

¹⁵ www.zdnet.com/blog/security/md5-password-scrambler-no-longer-safe/12317

¹⁶ <https://mail.python.org/pipermail/python-dev/2005-December/058850.html>

¹⁷ Try googling `0x7c6a180b36896a0a8c02787eeafb0e4c`. Better yet, create an MD5 hash of *your* password and google that. You may be surprised.

¹⁸ <http://docs.oracle.com/javase/6/docs/api/javax/crypto/spec/PBEKeySpec.html>

¹⁹ <http://grePCODE.com/file/repo1.maven.org/maven2/org.bouncycastle/bcprov-jdk15on/1.47/org/bouncycastle/crypto/generators/SCrypt.java>

²⁰ <https://www.bouncycastle.org/devmailarchive/msg13653.html>

²¹ <http://arstechnica.com/security/2013/09/long-passwords-are-good-but-too-much-length-can-be-bad-for-security/>

²² www.techweekeurope.co.uk/news/hardware-password-innovation-141263

²³ This is just an example. Never use SHA-256 for password storage; use PBKDF2 or scrypt!

²⁴ Assuming all uppercase and lowercase letters, plus digits.

²⁵ https://www.owasp.org/index.php/Password_Storage_Cheat_Sheet

²⁶ The Social Security number may not be good general advice for everyone (outside of banking) because it requires some handling precautions: www.parealtor.org/clientuploads/Legal/Statutes/SSNPrivacyAct.pdf

²⁷ Good security questions must be memorable, consistent, nearly universal, and safe. For more on choosing good security questions go here: https://www.owasp.org/index.php/Choosing_and_Using_Security_Questions_Cheat_Sheet.

²⁸ https://www.owasp.org/index.php/Forgot_Password_Cheat_Sheet

²⁹ www.wired.com/2011/06/rsa-replaces-securid-tokens

³⁰ <http://hueniverse.com/2009/04/explaining-the-oauth-session-fixation-attack/>

³¹ https://www.owasp.org/index.php/Session_Management_Cheat_Sheet

³² https://www.owasp.org/index.php/Authentication_Cheat_Sheet

³³ <https://shiro.apache.org/java-authentication-guide.html>